# IDAT7215
# Computer Programming for Product Development and Applications

## Lecture 3-2: Expert Systems and Introduction to PyKE Library

Dr. Zulfiqar Ali

# Outline

- What is Expert System
- Components of Expert System
- Applications of Expert System
- Introduction to PyKE
  - Family Example
  - Weather Example

# WHAT IS AN EXPERT SYSTEM ?




- An Expert System (ES) is an example of a knowledge-based system. It was introduced around 1965 at Stanford by *Edward Feigenbaum*.

- "ES is a computer program that uses artificial intelligence (AI) technologies to simulate the judgment and behavior of a human or an organization that has expert knowledge and experience in a particular field such as medical diagnosis, account, coding, and games, etc."

- ES gathers data by asking the user questions about the problem. An initial set of questions can lead to further questions depending on the user's responses.

- The ES reasons what questions it needs to ask, based on the knowledge it is given. It will use the responses from users to rule out various possibilities that will allow it eventually reach a decision or diagnosis.

# WHAT IS AN EXPERT SYSTEM ?




- We rely on expert systems to help us with knowledge and understanding in a particular field. We use them the same way as certain experts in our lives.

- For example, we will see a doctor if we have a health problem, or we will see a car mechanic if our car will not start.

- We can similarly use an expert system; it will ask us questions about our health problem or car problem and provide us with a diagnosis or course of action.

- ES is designed to try and replicate the judgment of a human that has expert knowledge in a certain field. By doing this they can be used to replace or assist a human expert.

# Why Develop Expert System

- Human expertise:
  - In short supply (few experts on task)
  - Expensive to develop or hire
  - Hard to get a hold of in a hurry
    - ✓ Expertise difficult to apply quickly (complex tasks)
    - ✓ Impractical to have humans constantly on call
  - Can be lost
    - ✓ Retirement/Leave job
    - ✓ Death

- ES serves the following functions with respect to expertise:
  - Document/preserve
  - Reproduce/make available
  - Teach/disseminate

- Computers do not get bored/tired/frustrated/scared

# Research Expert Systems

ELSEVIER

About Elsevier | Products & Solutions | Services | Shop & Discover

Home > Journals > Expert Systems with Applications




ISSN: 0957-4174

## Expert Systems with Applications

An International Journal

Publishing options: OA Open Access | S Subscription

Guide for authors | Track your paper | Order journal

Submit your paper

The Impact Factor of this journal is 8.665, ranking it 23 out of 276 in *Engineering, Electrical & Electronic*

With this journal indexed in 7 international databases, your published article can be read and cited by researchers worldwide

View articles

**Editor-in-Chief** > Editorial board



Prof. Dr. Binshan Lin, PhD

*Expert Systems With Applications* is a refereed international journal whose focus is on exchanging information relating to **expert** and **intelligent systems** applied in **industry**, **government**, and **universities** worldwide. The thrust of the journal is to publish papers dealing with the design, development, testing, implementation,

Read full aims & scope

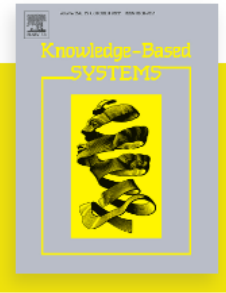
| CiteScore | Impact Factor | Top Readership | Publication Time |
|---|---|---|---|
| 12.2 | 8.665 | CN US GB | 1.4 weeks |

View historical data and other metrics on Journal Insights.

# Research Expert Systems

ScienceDirect

Journals & Books | Register | Sign in




## Knowledge-Based Systems

Supports *open access*

12
CiteScore

8.139
Impact Factor

Articles & Issues | About | Publish | Order journal | Search in this journal | Submit your article | Guide for authors

Latest issue

**Volume 256**

*In progress*

28 November 2022

### About the journal

*Knowledge-based Systems* is an international and interdisciplinary journal in the field of artificial intelligence. The journal will publish original, innovative and creative research results in the field, and is designed to focus on research in knowledge-based and other artificial intelligence ...

View full aims & scope

1.2 weeks
Publication Time

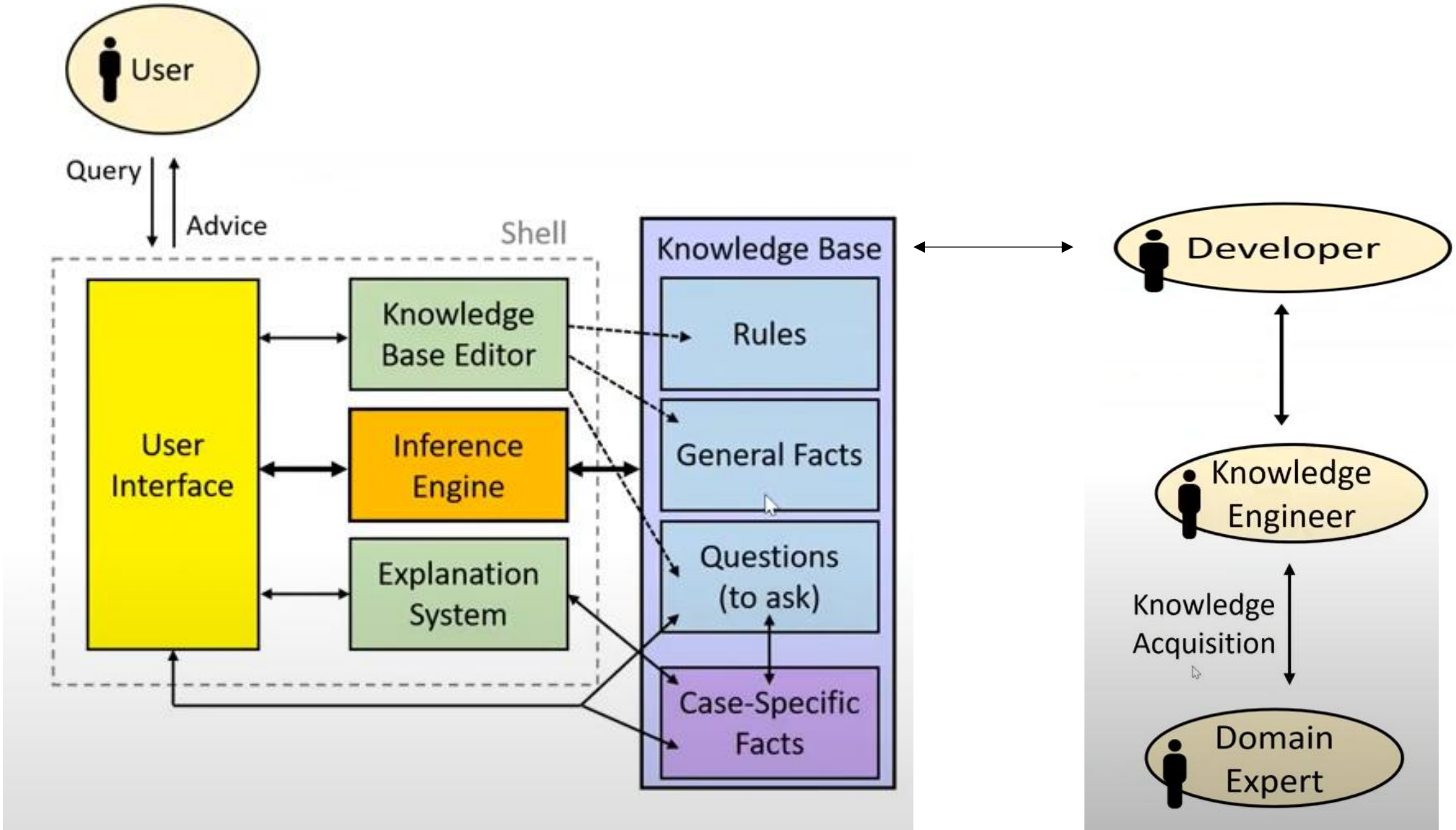
View all insights

FEEDBACK

# ES Building Process

- First attempt unlikely to be very successful
  - Expert generally finds it difficult to express exactly what knowledge and rules they use to solve a problem.
  - Common sense/subconscious knowledge
    - ✓ Seems so obvious, they don't bother mentioning it.
- Knowledge Acquisition:
  - Gather/extract relevant knowledge.
- Knowledge Engineering
  - Build the expert system knowledge base
- Initial prototype developed and shown to expert
  - Check the system's performance and give feedback.
  - Knowledge base refinement.
- Iterative development from prototype
  - Feedback from experts and end users of the system.

# The General Design of an Expert System



# Knowledge Base

- The developers of the expert system will interview a collection of experts to build the database of knowledge.
- They will look to gain two types of knowledge from the experts.
- The first is factual knowledge. This is the knowledge that is widely shared.
- The second type is heuristic knowledge. This is the knowledge that is more personal and is acquired through a range of experiences and reasoning.

# Knowledge Base

- Once the knowledge base is built, it can be used by the expert system to inform the question sit needs to ask and assist in providing the results.

- Part of the knowledge base is the rule base. The rule base is a set of rules that will be used to produce an output or decision by the expert system. These rules are used by the inference engine as a base for reasoning, to obtain a solution to a problem or a decision.

- Each rule will contain two parts, the IF and the THEN. A rule can also have multiple IF parts that will be joined together by Boolean operators including AND and OR.

- Most ES will have a knowledge-based editor built into them. This allows the knowledge base to be checked for errors and edited and updated when needed.

# Inference Engine

- It will ask the user questions and based on their answer it will follow a line of logic. This may then lead to further questions and eventually a final result.

- The inference engine is mostly a problem-solving tool. It organizes and controls steps to solve the problem. To do this it often uses a chaining method. It chains together what is known as IF-THEN rules to form a line of reasoning.

- The IF part of the rule is a condition, for example, IF  I am hungry. The Then part of the rule is an action, for IF I am hungry THEN I will need to eat.

# Inference Engine

- There are two main methods that can be used to obtain a result. The appropriate one will depend on whether the expert system is designed to produce a final result i.e., a diagnosis or course of action, or if it begins with a known conclusion i.e., goal.

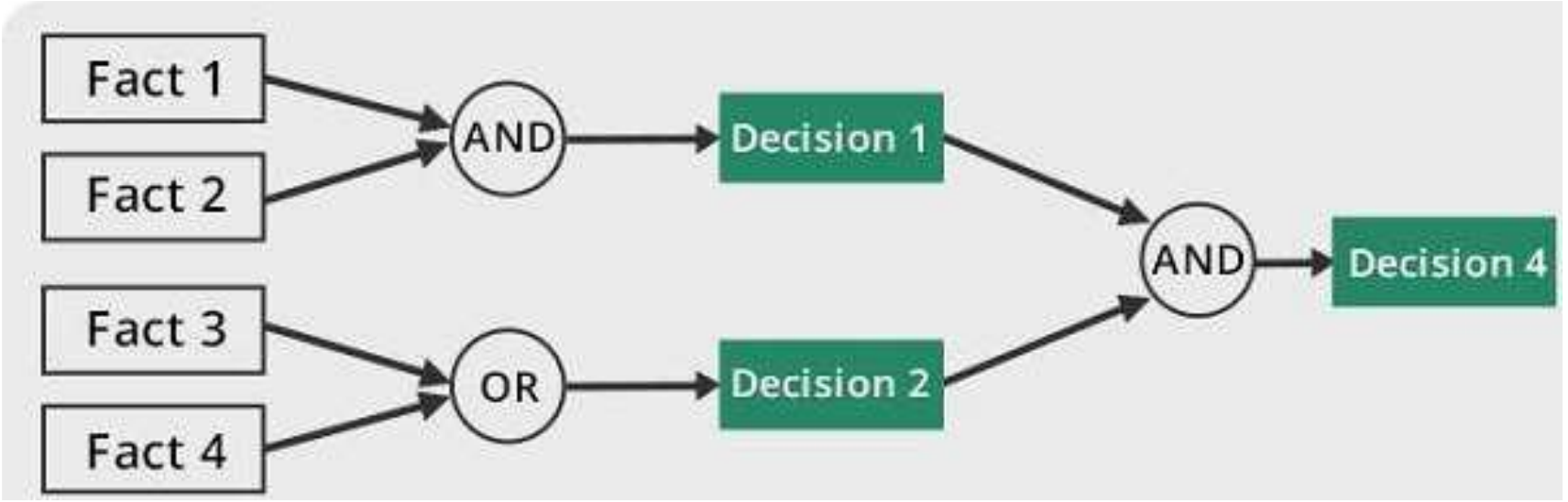
- If the process starts with a set of conditions and chaining moves towards a final conclusion, this is called forward chaining.

- If the process starts with a known conclusion, but the path to it is unknown, the chaining is called backward chaining.

# Inference Engine-Forward Chaining

- In a forward chaining system, the ES will take the data input and match it to the knowledge and rules it contains. It will keep doing this until I can reach an end goal or outcome. Therefore, we can say a forward chaining system is data-driven.

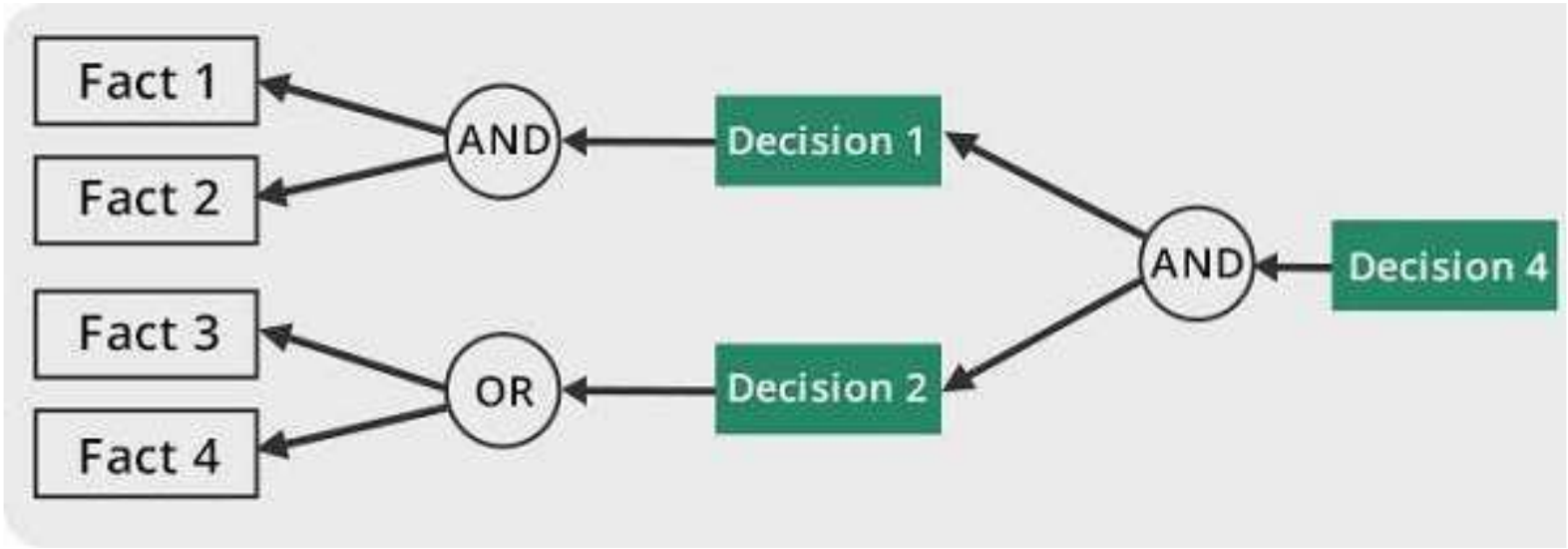
- Data is gathered about the problem and then the system infers what it can from the data to reach a conclusion.



# Inference Engine-Backward Chaining

- With this strategy, an expert system finds out the answer to the question, “Why

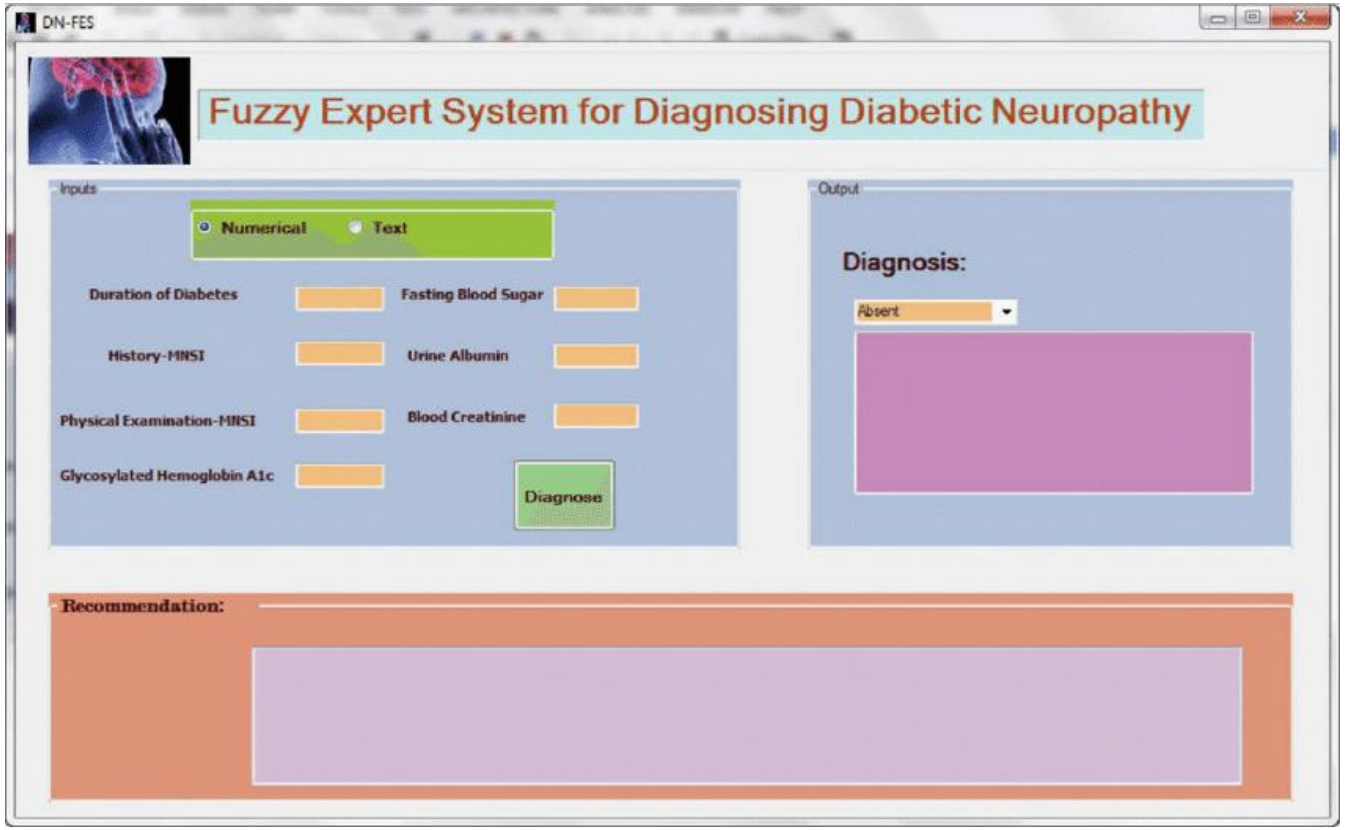
- On the basis of what has already happened, the Inference Engine tries to find out which conditions could have happened in the past for this result.



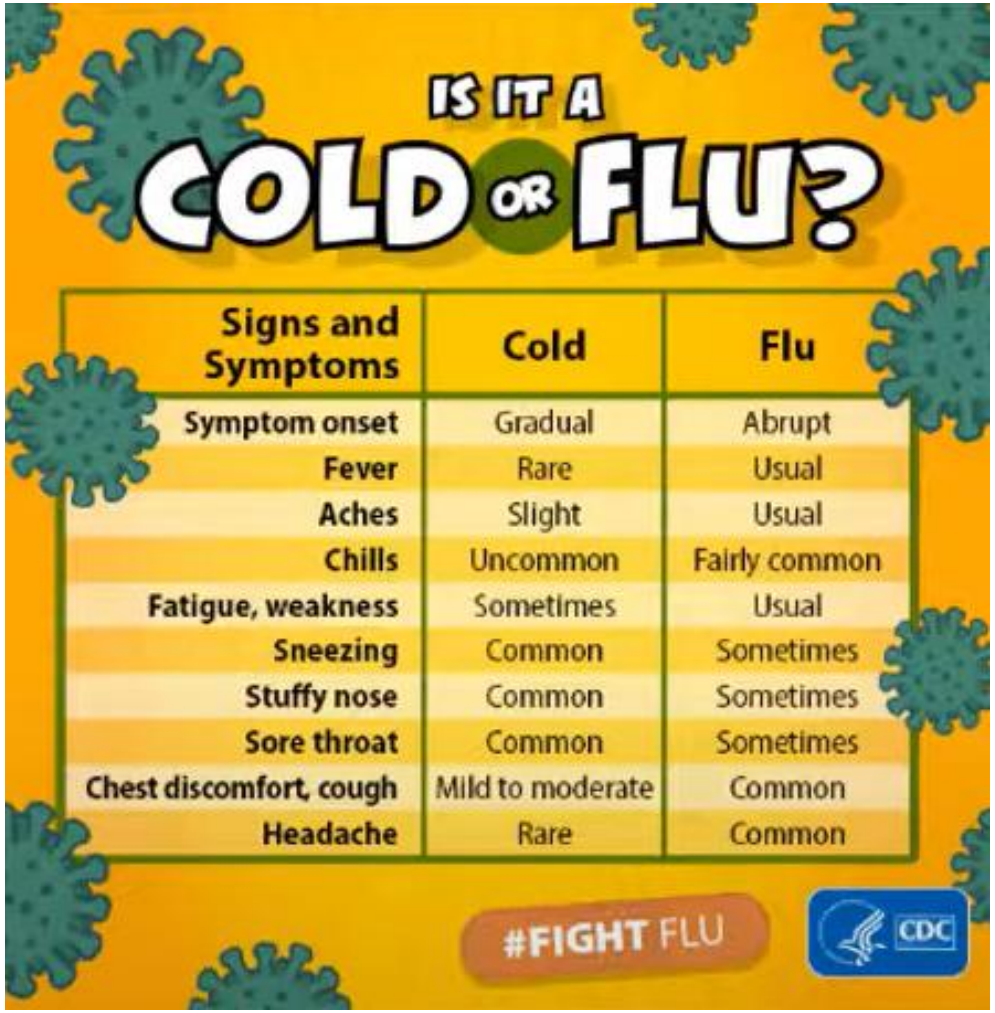
# User Interface

- The user interface is the way the user interacts with the expert system.

- They are often graphical in nature and have a range of selection processes or typing methods to allow the user to provide responses.



# Example




| Signs and Symptoms | Cold | Flu |
|---|---|---|
| Symptom onset | Gradual | Abrupt |
| Fever | Rare | Usual |
| Aches | Slight | Usual |
| Chills | Uncommon | Fairly common |
| Fatigue, weakness | Sometimes | Usual |
| Sneezing | Common | Sometimes |
| Stuffy nose | Common | Sometimes |
| Sore throat | Common | Sometimes |
| Chest discomfort, cough | Mild to moderate | Common |
| Headache | Rare | Common |

```
Flu=0
Cold=0

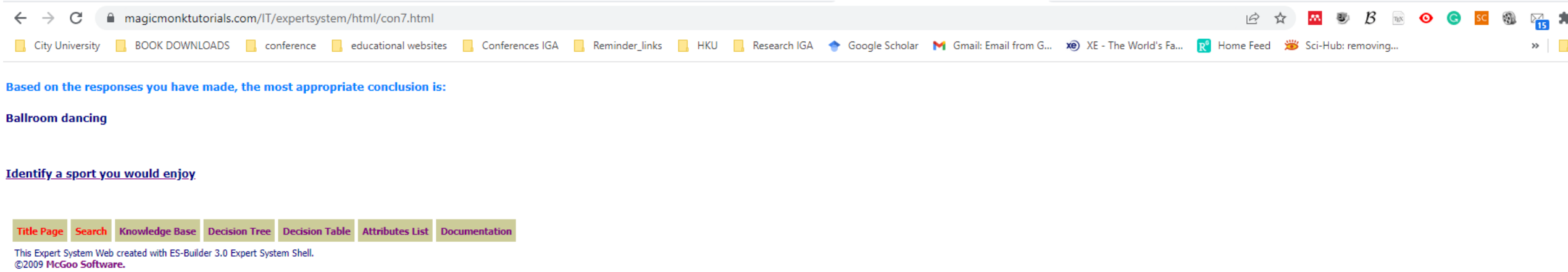
IF onset="Abrupt"  flu=flu+1
            ELSE cold=cold+1
IF fever == TRUE  flu=flu+1
            ELSE cold = cold +1
IF chills = TRUE flu = flu +1
            ELSE cold=cold+1

…

IF fever> cold print( "It's probably the flu")
            ELSE print("It's probably a cold")
```

# Demo: Identify a sport you would enjoy

- Try this via clicking the weblink




https://magicmonktutorials.com/IT/expertsystem/html/con7.html

# Shallow Expert System vs Deep Expert System

- Shallow Expert Systems are first-generation architecture expert systems, and which lack causal knowledge (knowledge of the underlying causes and effects in a system) while Deep Expert Systems are second-generation architecture expert systems that alleviate first-generation architecture expert systems' limitations.

- Shallow Expert Systems use surface knowledge which can only represent expertise while Deep Expert Systems use deep knowledge which can represent complex structured objects using object models.

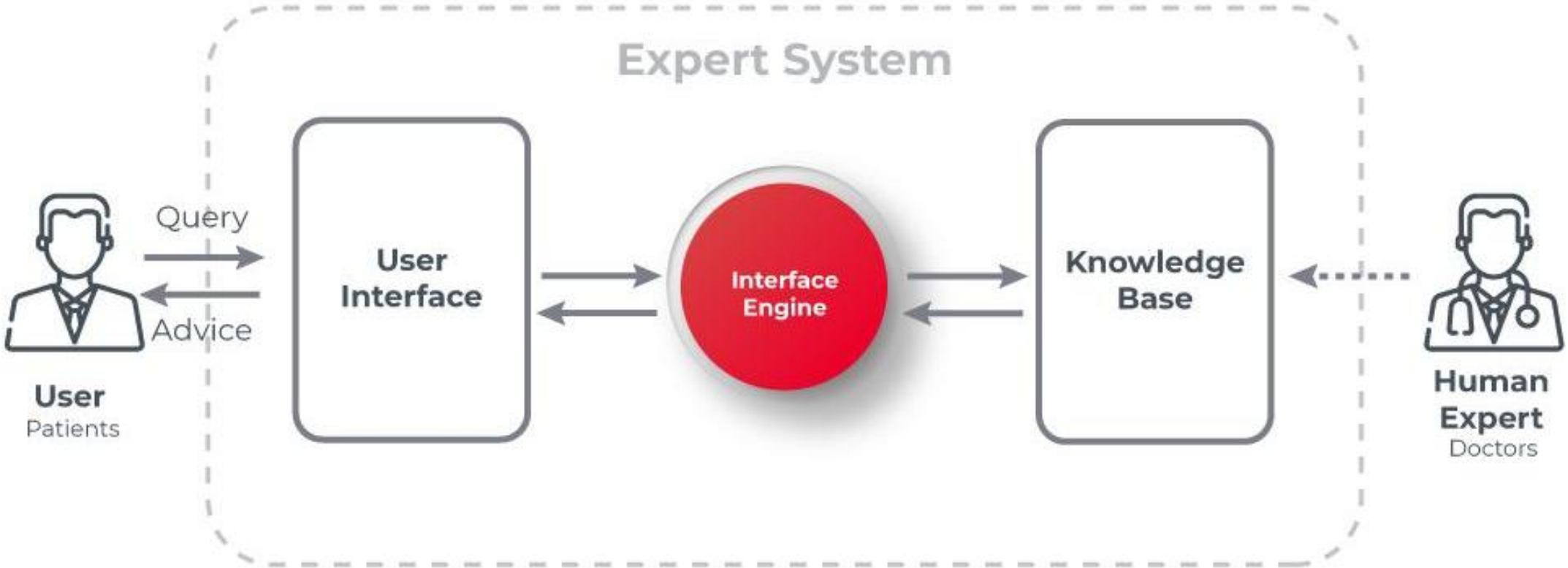
- Shallow Expert Systems use rule-based reasoning while Deep Expert Systems combine both rule-based reasoning and model-based reasoning.

- Shallow Expert Systems use heuristics or heuristic rules or rules of thumb, which do not guarantee optimal solutions while Deep Expert Systems use the A* algorithm which guarantees optimality by finding solutions in reasonable computation time, in terms of, number of transformations and steps of a search.

# Applications of Expert System

- Medical diagnosis.
- Providing financial advice.
- Manufacturing Production & Maintenance
- Troubleshooting computer and printer issues.
- Identifying items, for example, plants and birds.
- Using a telephone help desk
- Playing chess

# **MYCIN**: Early Medical Diagnostic System (1970s)



# MYCIN: Early Medical Diagnostic System (1970s)

- MYCIN was the earliest designed expert system at Stanford University in the 1970s.
- Its job was to diagnose and recommend treatment for certain blood infections.
- MYCIN was written in LISP Programming Language.
- ~ 600 rules to diagnose the presence of certain bacteria and recommend antibiotics.
- Combined rules in a more sophisticated way (Bayesian inference)
- Never used in practice due to ethical concerns.
- Inspired the creation of development environments for others who DID make expert systems that were put into use.

# MYCIN: Early Medical Diagnostic System (1970s)

- MYCIN represented its knowledge as a set of IF-THEN rules with certainty factors. The following is an English version of one of MYCIN's rules:

  IF the infection is primary-bacteremia

  AND the site of the culture is one of the sterile sites

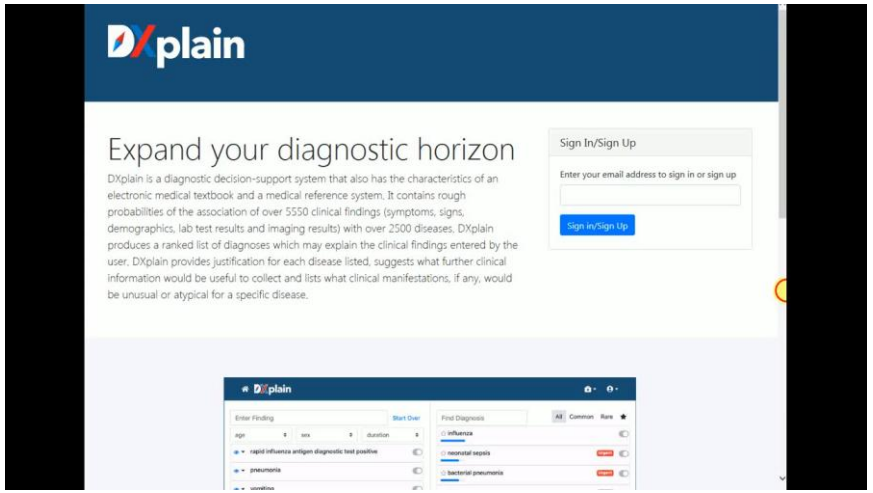
  AND the suspected portal of entry is the gastrointestinal tract

  THEN there is suggestive evidence (0.7) that infection is bacteroid.

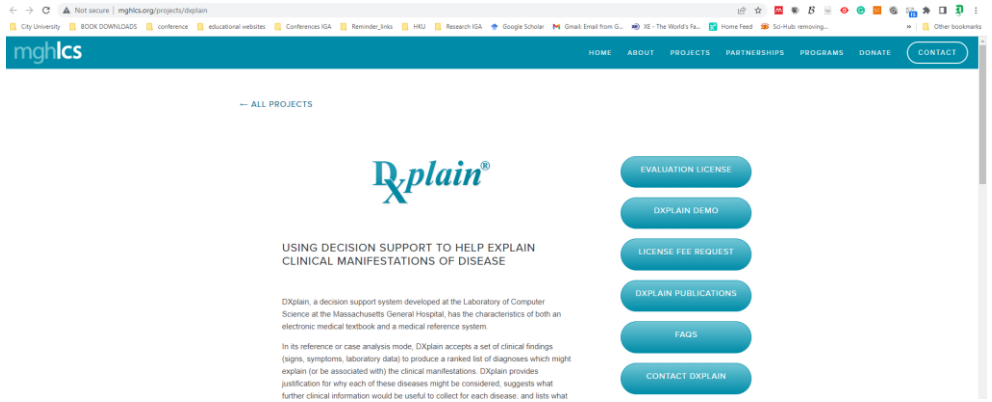
- The 0.7 is roughly the certainty that the conclusion will be true given the evidence. If the evidence is uncertain, the certainties of the bits of evidence will be combined with the certainty of the rule to give the certainty of the conclusion.

# Other Medical Expert System

- **DXPLAIN:** It is used for diagnosis. DXplain provides justification for why each of these diseases might be considered, suggests what further clinical information would be useful to collect for each disease, and lists what clinical manifestations.



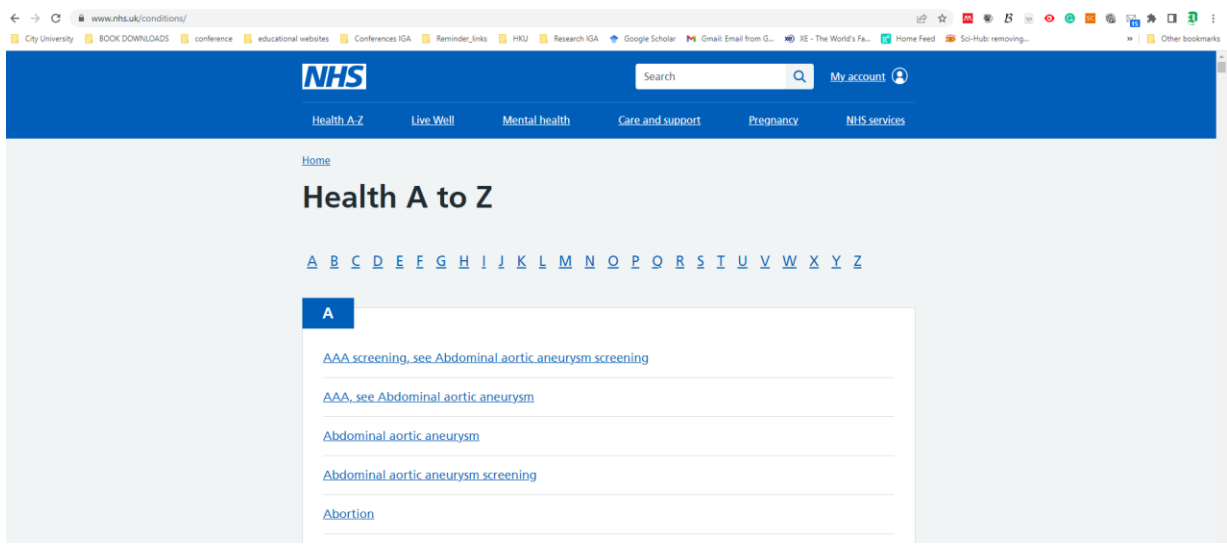

- **CaDet:** It is for early cancer detection. Clinical data related to early cancer detection and to cancer risk factors was collected and incorporated into the database, together with heuristic rules for evaluating this data.




http://www.mghlcs.org/projects/dxplain

# Example- National Health Service (NHS) UK

- The National Health Service (NHS) is the medical system in the UK. The NHS has a website that allows users to enter the symptoms of their illness or ailment and it will provide them with a possible diagnosis. This may help a person to understand which possible illnesses they may have. It can be used by professionals within the NHS to double-check a diagnosis they are giving, or to aid them with any areas of weakness they have in their medical knowledge.




https://www.nhs.uk/conditions/

# Banking and Financial Sector:

- An Expert System that helps bank managers in making decisions on granting loans.
- An Expert System that advises bank managers in giving housing loans.
- An Expert System that advises insurance companies on the risks involved in insuring a customer or a company.
- An Expert System that helps banks decide on whether a customer is entitled to a credit card or not.
- An Expert System that identifies computer fraud and controls it.

# Application of Expert System Production Scheduling

- Due to the changing production style and highly dynamic variations in production requirement Expert System Manufacturing is one of the widely applied used areas.

- One of the most common uses of ES in manufacturing is Scheduling and Production planning.

- The first application of an Expert System in job scheduling is **Intelligent Scheduling and Information System** (ISIS), 1984.

# Application of Expert System Production Scheduling

- Some Scheduler using Expert System technology

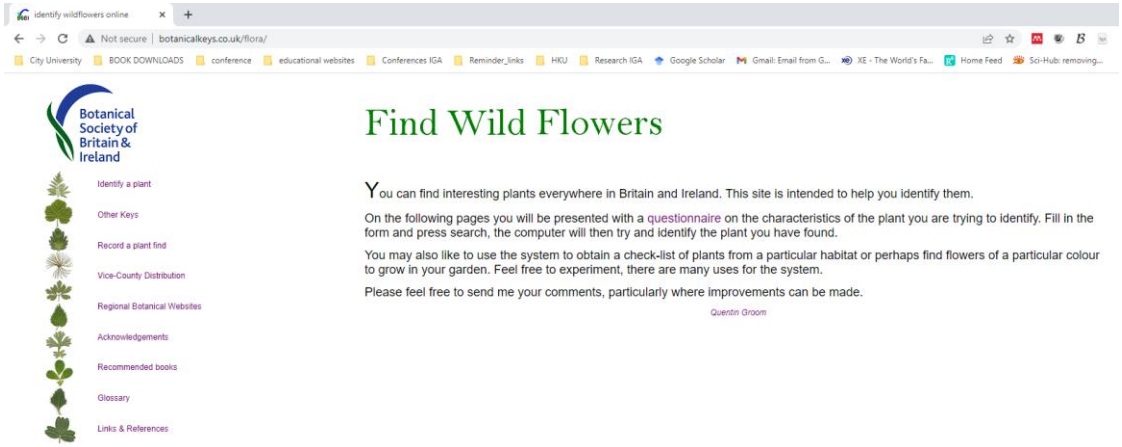
| *Project* | *Group* | *Domain* |
|---|---|---|
| MASCOT | Parunak, 1993<br>ITI | Manufacturing scheduling and control |
| DAS | Burke and Prosser, 1991<br>University of Strathclyde | Manufacturing scheduling |
| ABACUS | McEleney *et al.*, 1998<br>UCB, UMIST | Manufacturing scheduling |
| MetaMorph II | Shen *et al.*, 1998<br>University of Calgary | Intelligent manufacturing production |
| SFA | Parunak, 1996<br>NCMS | Manufacturing scheduling and control |
| A case based expert system for generative computer-aided process planning with manufacturing uncertainty | Wong, 1997<br>MSERC | Manufacturing planning |
| IAO | Kwok and Norrie, 1994<br>University of Calgary | Intelligent manufacturing |

# Application of Expert System Production Scheduling

- Advantages of using Expert System in Manufacturing According to *Byrd (1995)*
  - Reduction in time to complete the tasks
  - Increase in production
  - Reduction in stuff
  - More effective use of resource

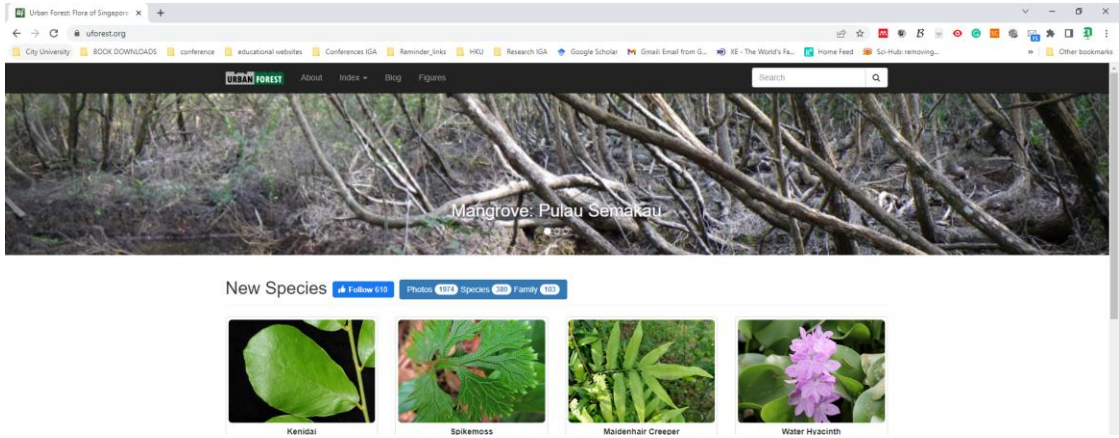
# Botanical Keys –UK ( Plant identification)

- The website Botanical keys allow a user to enter a range of characteristics about a plant. The system will then identify what the plant is. Users can use this to find out what plants are growing in their garden or identify a plant they have seen when out on a walk in the countryside or through a park.




http://www.botanicalkeys.co.uk/flora/

- Urban Forest is a non-profit website in Singapore that contains information about flora. From Singapore and Southeast Asia-users can visit the site to help them with plant identification.

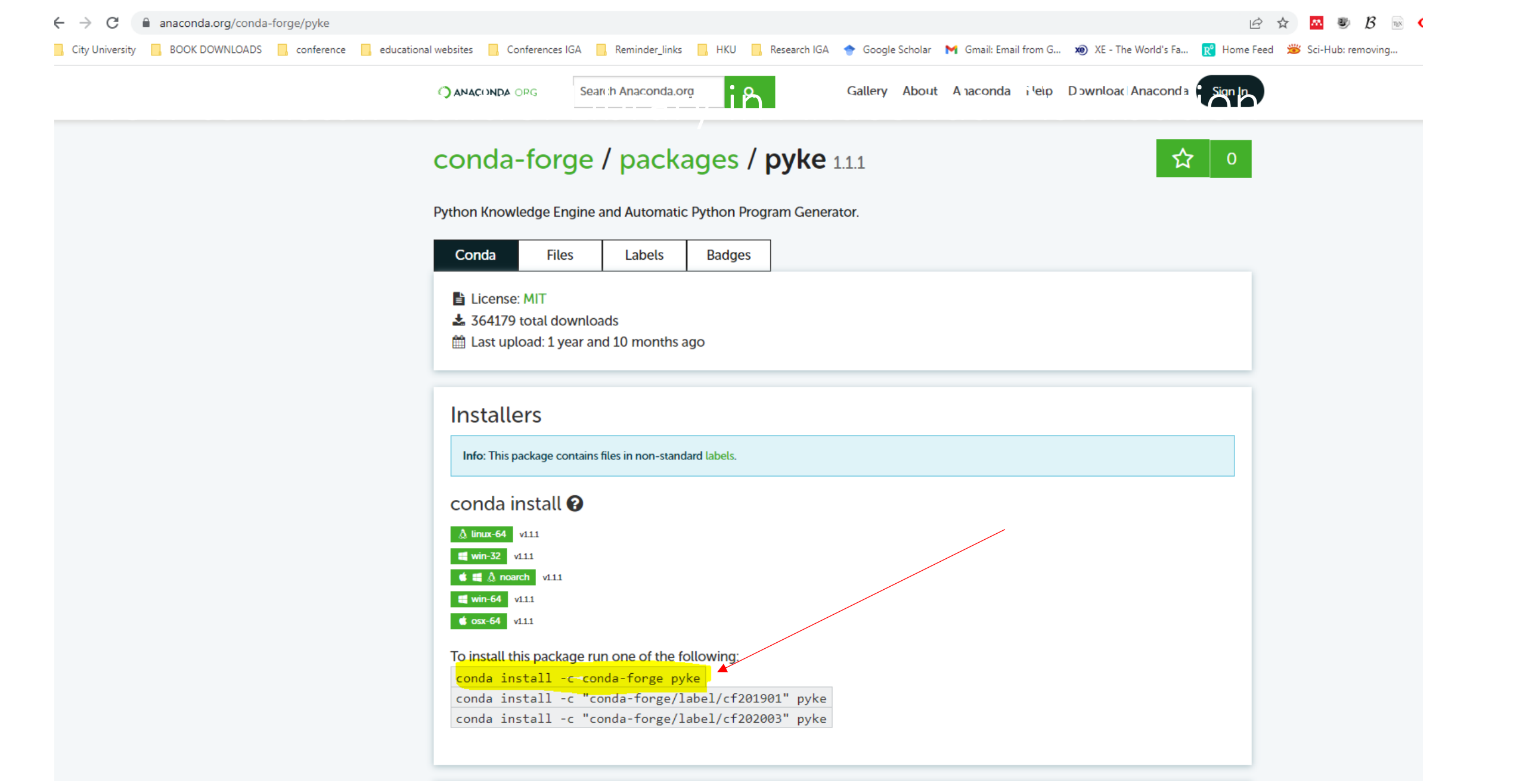



https://uforest.org/

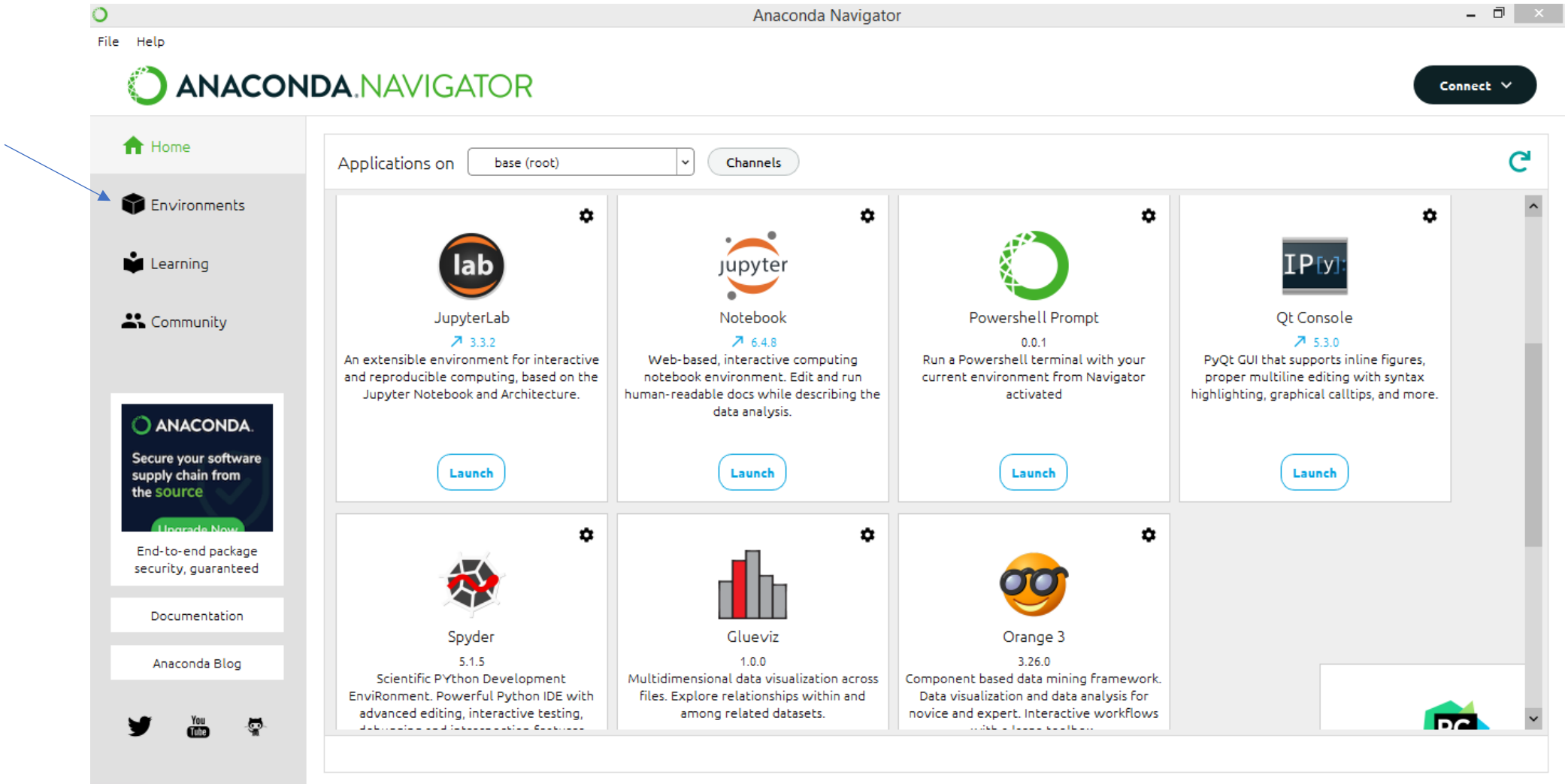
# Expert System

| Advantages | Disadvantages |
| --- | --- |
| ▪ They provide answers to questions that are outsides a user's knowledge and experience. | ▪ They do not have the addition of common sense that humans have. This means that their response can be a logical one and cannot have a creative approach. |
| ▪ They can aid professionals in areas where their knowledge and experience are a little weaker. | ▪ Errors in the knowledge base or inference engine will produce errors in the results. Therefore, they are only as good as the data and rules they are given. |
| ▪ As they use a logical process they are consistent in the answers that they give. | ▪ They are not able to automatically adapt to changing environments. This requires the knowledge base to be changed. |
| ▪ They will not forget to ask a question, as all the rules and knowledge is put into place to make sure that all data needed is gathered. | ▪ They are expensive to produce as they require a great deal of time and effort from experts and developers. |

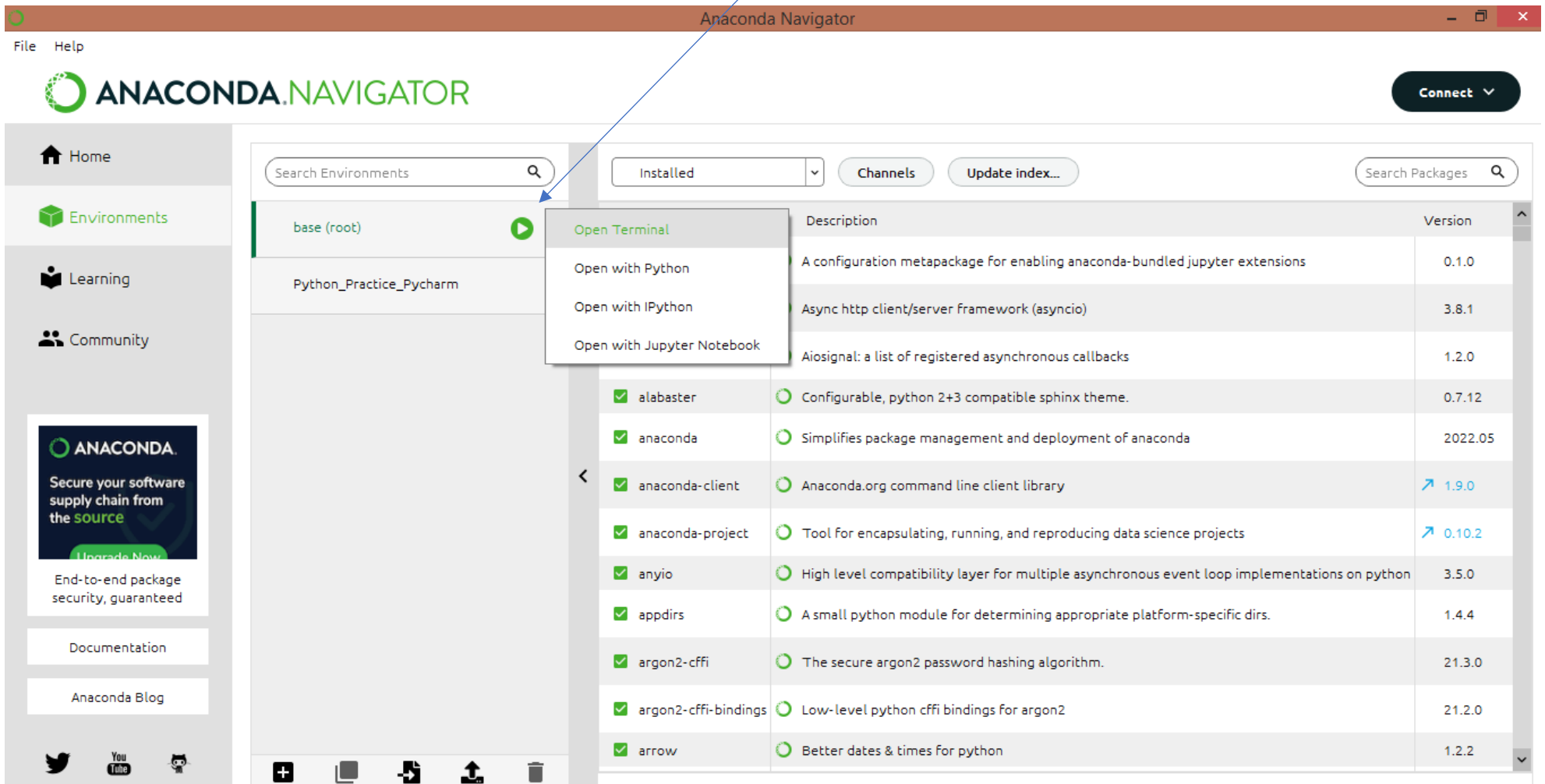
# Installing Pyke library in the Anaconda Distribution

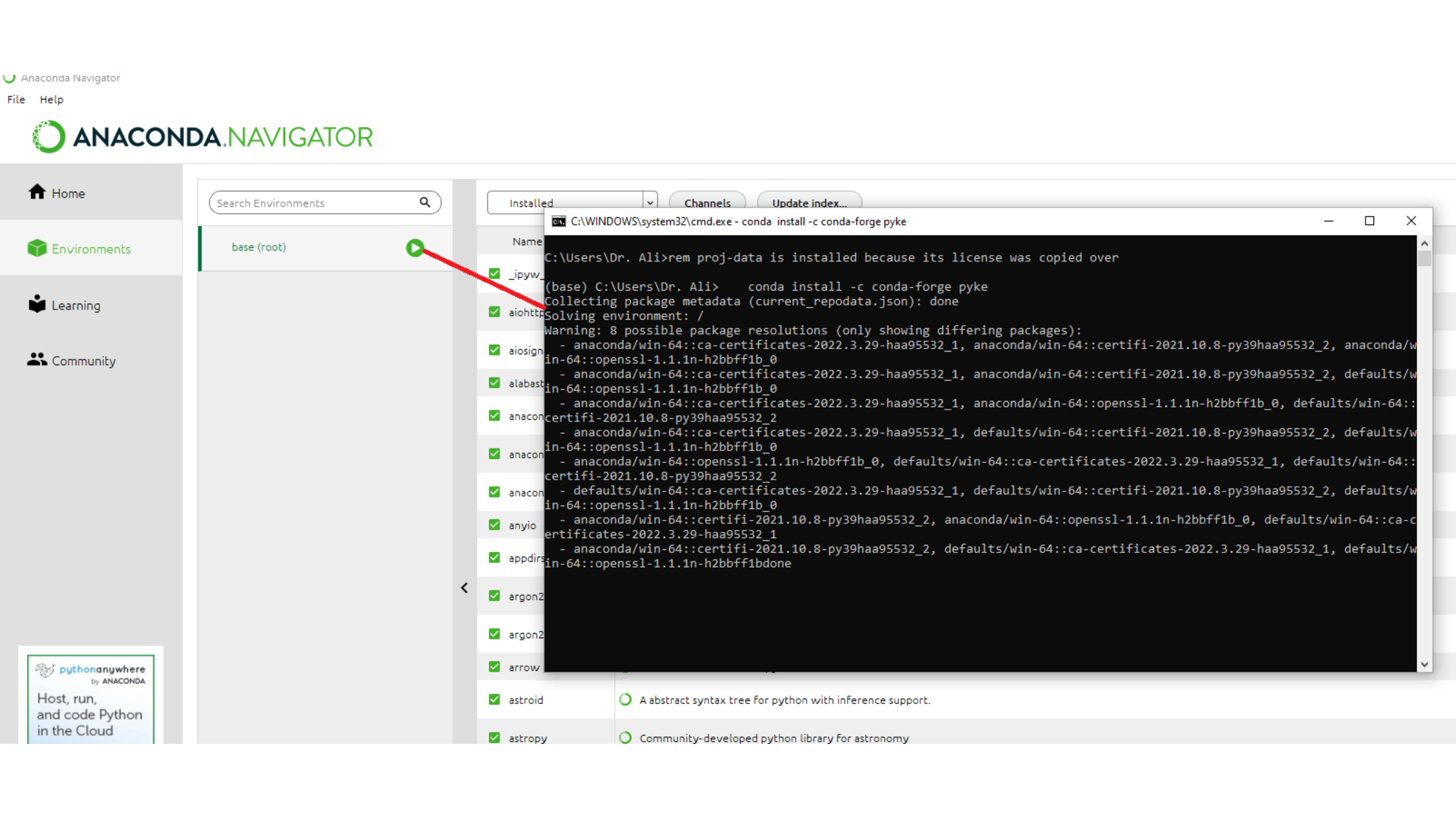


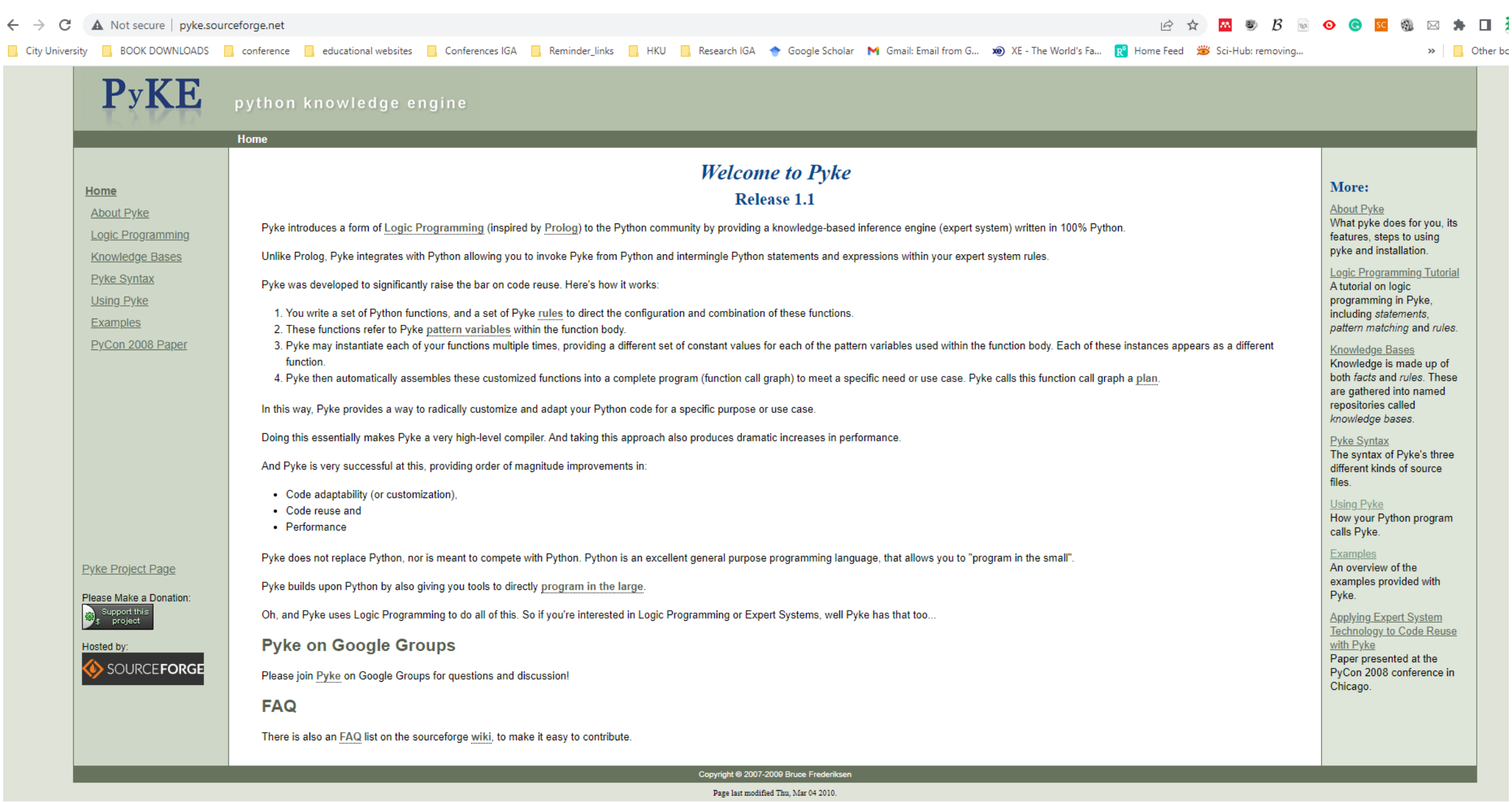

http://pyke.sourceforge.net/


Anaconda Navigator
File
Help
ANACONDA.NAVIGATOR
Connect
Home
Environments
Learning
Community
Applications on
base (root)
Channels
JupyterLab
3.3.2
An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.
Launch
Notebook
6.4.8
Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.
Launch
Powershell Prompt
0.0.1
Run a Powershell terminal with your current environment from Navigator activated
Launch
Qt Console
5.3.0
PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.
Launch
Spyder
5.1.5
Scientific PYthon Development EnviRonment. Powerful Python IDE with advanced editing, interactive testing,
Glueviz
1.0.0
Multidimensional data visualization across files. Explore relationships within and among related datasets.
Orange 3
3.26.0
Component based data mining framework. Data visualization and data analysis for novice and expert. Interactive workflows
ANACONDA.
Secure your software supply chain from the source
Upgrade Now
End-to-end package security, guaranteed
Documentation
Anaconda Blog

Open Terminal




Anaconda Navigator
File Help
ANACONDA.NAVIGATOR
Home
Environments
Learning
Community
Search Environments
base (root)
Installed
Channels
Update index...
Name
C:\WINDOWS\system32\cmd.exe - conda install -c conda-forge pyke
C:\Users\Dr. Ali>rem proj-data is installed because its license was copied over
(base) C:\Users\Dr. Ali> conda install -c conda-forge pyke
Collecting package metadata (current_repodata.json): done
Solving environment: /
Warning: 8 possible package resolutions (only showing differing packages):
- anaconda/win-64::ca-certificates-2022.3.29-haa95532_1, anaconda/win-64::certifi-2021.10.8-py39haa95532_2, anaconda/win-64::openssl-1.1.1n-h2bbff1b_0
- anaconda/win-64::ca-certificates-2022.3.29-haa95532_1, anaconda/win-64::certifi-2021.10.8-py39haa95532_2, defaults/win-64::openssl-1.1.1n-h2bbff1b_0
- anaconda/win-64::ca-certificates-2022.3.29-haa95532_1, anaconda/win-64::openssl-1.1.1n-h2bbff1b_0, defaults/win-64::certifi-2021.10.8-py39haa95532_2
- anaconda/win-64::ca-certificates-2022.3.29-haa95532_1, defaults/win-64::certifi-2021.10.8-py39haa95532_2, defaults/win-64::openssl-1.1.1n-h2bbff1b_0
- anaconda/win-64::openssl-1.1.1n-h2bbff1b_0, defaults/win-64::ca-certificates-2022.3.29-haa95532_1, defaults/win-64::certifi-2021.10.8-py39haa95532_2
- defaults/win-64::ca-certificates-2022.3.29-haa95532_1, defaults/win-64::certifi-2021.10.8-py39haa95532_2, defaults/win-64::openssl-1.1.1n-h2bbff1b_0
- anaconda/win-64::certifi-2021.10.8-py39haa95532_2, anaconda/win-64::openssl-1.1.1n-h2bbff1b_0, defaults/win-64::ca-certificates-2022.3.29-haa95532_1
- anaconda/win-64::certifi-2021.10.8-py39haa95532_2, defaults/win-64::ca-certificates-2022.3.29-haa95532_1, defaults/win-64::openssl-1.1.1n-h2bbff1bdone
_ipyw
aiohttp
aiosign
alabast
anacon
anacon
anacon
anyio
appdirs
argon2
argon2
arrow
astroid
A abstract syntax tree for python with inference support.
astropy
Community-developed python library for astronomy
pythonanywhere by ANACONDA
Host, run, and code Python in the Cloud

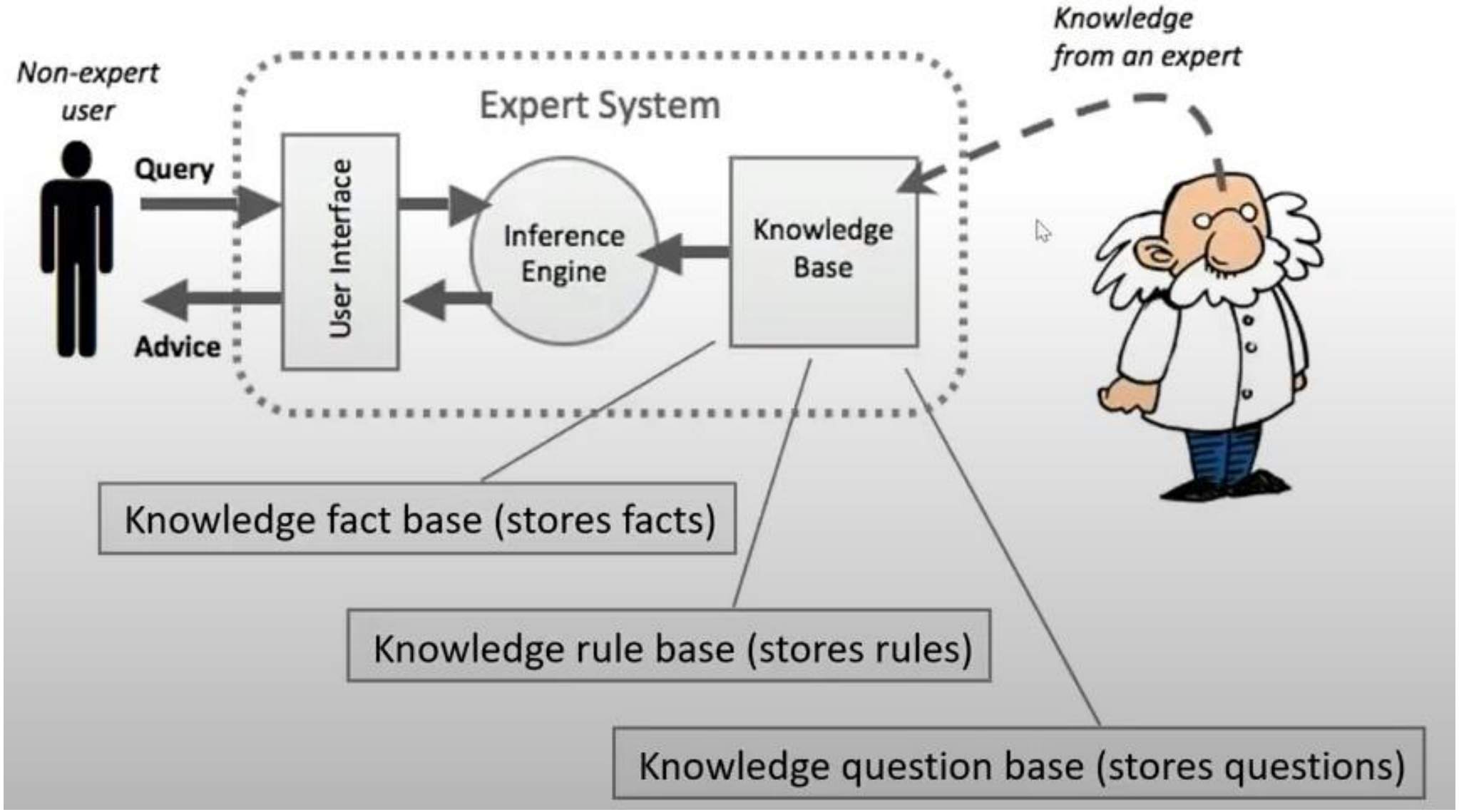



http://pyke.sourceforge.net/

# What is PyKE

- Python Knowledge Engine (PyKE)
- An expert system interpreter and coding framework that we will use in this course to learn how to build and run a simple expert system.
- Integrates a form of logic programming into python.
  - Provides an inference engine that :
  - Applies rules to facts to establish additional facts (forward chaining)
  - And/or prove goals (backward chaining)
  - Optionally assemble Python functions into customized call graphs, called plans..
- http://pyke.sourceforge.net/about_pyke/index.html

# What is PyKE


Knowledge
from an expert
Non-expert
user
Expert System
Query
User Interface
Inference
Engine
Knowledge
Base
Advice
Knowledge fact base (stores facts)
Knowledge rule base (stores rules)
Knowledge question base (stores questions)

# Why PyKE

- PyKE was chosen because it operates exclusively in Python (and unlikely PyCLIPS, it works in Python 3).
- Probably the most commonly used expert system framework today is called CLIPS, however, this is a C coding language interpreted system
  - If you want to (optionally) check out CLIPS, see the link below
  - Link: CLIPS website https://www.clipsrules.net/

# Using Pyke- The simples use of PyKE

- IDAT7215 Lab 3 Tutorial-  PyKE Installation and Family Relations Expert System Test.ipynb

## Pyke Instructions

1. Download pyke3-1.1.1.zip from https://sourceforge.net/projects/pyke/files/pyke/1.1.1/
2. Unzip pyke3-1.1.1.zip (if you are unfamiliar with how to unzip a file see https://www.wikihow.com/Unzip-a-File)
3. Check the unzipped folder to ensure its contents include folders by the name of (build, doc, examples, experimental, pyke, Test) along with a number of other files including README.txt. (Note when working with newly installed code, the README file includes important information on how to get started with and run the code)
4. Copy the folder with these PyKE files to where ever you wish to work from (i.e. your working directory).
5. Take a look at the pyke example folder '/examples/family_relations/'.
    - Within that folder you should see 7 files including 'driver.py '
        - There are three kinds of Pyke source files:
            - .kfb files define fact bases
            - .krb files define rule bases
            - .kqb files define question bases
    - Open and read the README files in this family_relations folder for an explanation of the different files and a set of examples that may be run.
6. Copy your unique path to this '/family_relations/' into the 'sys.path.append()' command in subtask 3. I've left how I did this on my computer as an example.
7. If these steps have been completed successfully then you should be able to run the commands in the cells below...

# Using Pyke- The simples use of Pyke

- Step 1: Create an engine object
  - From pyke import knowledge_engine
  - my_engine = knowledge_engine.engine(__file__)
- Step 2: Activate rule bases
  - my_engine.activate(‘rule_base_file)
- Step 3: Prove goals
  - from pyke import goal
  - my_goal=goal.compile(‘goal)

# Load PyKE Driver

In [1]:
```
#Run this cell
import sys
# The following command points your notebook to the location of a folder outside your working directory that you want to import.
sys.path.append('C:/Users/Dr. Ali/Lab 04-Expert System/pyke3-1.1.1/pyke-1.1.1/examples/family_relations')
#sys.path.append('yourpath')
import driver
```

driver.py

```
import contextlib
import sys
import time

from pyke import knowledge_engine, krb_traceback, goal

# Compile and load .krb files in same directory that I'm in (recursively).
engine = knowledge_engine.engine(__file__)

fc_goal = goal.compile('family.how_related($person1, $person2, $relationship)')

def fc_test(person1 = 'bruce'):
    '''
        This function runs the forward-chaining example (fc_example.krb).
    '''
    engine.reset()      # Allows us to run tests multiple times.

    start_time = time.time()
    engine.activate('fc_example')  # Runs all applicable forward-chaining rules.
    fc_end_time = time.time()
    fc_time = fc_end_time - start_time

    print("doing proof")
    with fc_goal.prove(engine, person1=person1) as gen:
        for vars, plan in gen:
            print("%s, %s are %s" % \
                    (person1, vars['person2'], vars['relationship']))
    prove_time = time.time() - fc_end_time
    print()
    print("done")
    engine.print_stats()
    print("fc time %.2f, %.0f asserts/sec" % \
          (fc_time, engine.get_kb('family').get_stats()[2] / fc_time))
```

# Test PyKE: Family-Forward Chaining

### Test 1

In [2]:
```
# Run this cell
driver.fc_test('michael_k')
```

```
doing proof
michael_k, amanda are ('father', 'daughter')
michael_k, tammy are ('father', 'daughter')
michael_k, crystal are ('father', 'daughter')

done
family: 9 fact names, 94 universal facts, 6920 case_specific facts
fc_example: 20 fc_rules, 6772 triggered, 892 rerun
fc_example: 0 bc_rules, 0 goals, 0 rules matched
            0 successes, 0 failures
fc time 0.35, 19964 asserts/sec
```

### Test 2

In this second test, we simply demonstrate what happens when a name is given that doesn't appear in the fact base.

In [3]:
```
# Run this cell
driver.fc_test('ryan')
```

```
doing proof

done
family: 9 fact names, 94 universal facts, 6920 case_specific facts
fc_example: 20 fc_rules, 6772 triggered, 892 rerun
fc_example: 0 bc_rules, 0 goals, 0 rules matched
            0 successes, 0 failures
fc time 0.33, 21235 asserts/sec
```

# Knowledge Bases- Fact Bases and Rule bases

**1. Fact base** stores facts in the form of a knowledge fact base (.kfb) file

- Two types of facts:
  - Case Specific facts: will be deleted when the inference engine is rested to prepare for another run of the inference engine.
  - Universal facts: never deleted.

**2. Rule base** stores rule in the form a of knowledge rule base (.krb) file.

- A single rule base may contain both forward chaining rules and backward chaining rules.

# Knowledge Bases- Fact Bases and Rule bases

- **Question base** stores questions for end users in the form of a knowledge question base (.kqb) file

- The .kqb file contains all the information about the question needed to ask the question, validate the answer, and output the appropriate review text.

- Questions can be of various types (e.g., yes/no, true/false, multiple choice)

- Example Question

- Who_father($ans)

Who is user father?

---

$ans = select_1

1. Joe  2. Steve  3. Bill  4. Dave

# PyKE: What is statement

- A statement is a generalized expression of knowledge, also just called a **fact**.
- Three components
  1. Name of a knowledge base
  2. Name of knowledge entity
     - The relationship between the arguments.
  3. State arguments
     - Can be any simple python data value (numbers, strings, None, True or False) or tuples of these values (including nested tuples)
     - Change of order arguments would change the meaning of a statement

family.son_of(son, father, mother)

family.son_of(Bruce, Thomas, Norma)

Semantics: “Bruce is the son of Thomas (his father) and Norma( his mother).”

# PyKE: Pattern Matching

- Literal Patterns:
  - Look exactly like data
  - Only match themselves.
  - Serve as inputs to Pyke.
  - Used when asking " Is statement X true?"

```
family.son_of(son, father, mother)
```

- Pattern Variables:
  - Serve as output parameters.
  - Start with a $.
  - Either bound to a value (literal pattern) or unbound.
  - Used when asking " Who are the sons of Thomas and Norma?"

```
family.son_of($son, father, mother)
```

- Anonymous pattern variables:
  - Start with $ and an underscore ($_).
  - Never bound to values.
  - Used when asking "Who are Norma's sons?" and don't care who are father is.

```
family.son_of($son, $_father, mother)
```

# PyKE: Rules

- Rules have two parts:
  - If part (containing a list of statements called the premises)
  - The part (containing a list of statements called the conclusions)

- Each of these if and the parts contain **one or more** facts or goals

Syntax: If [A], [B], and [C], Then [D] and [E]

Semantics: If A, B and C are true, then D and E are true.

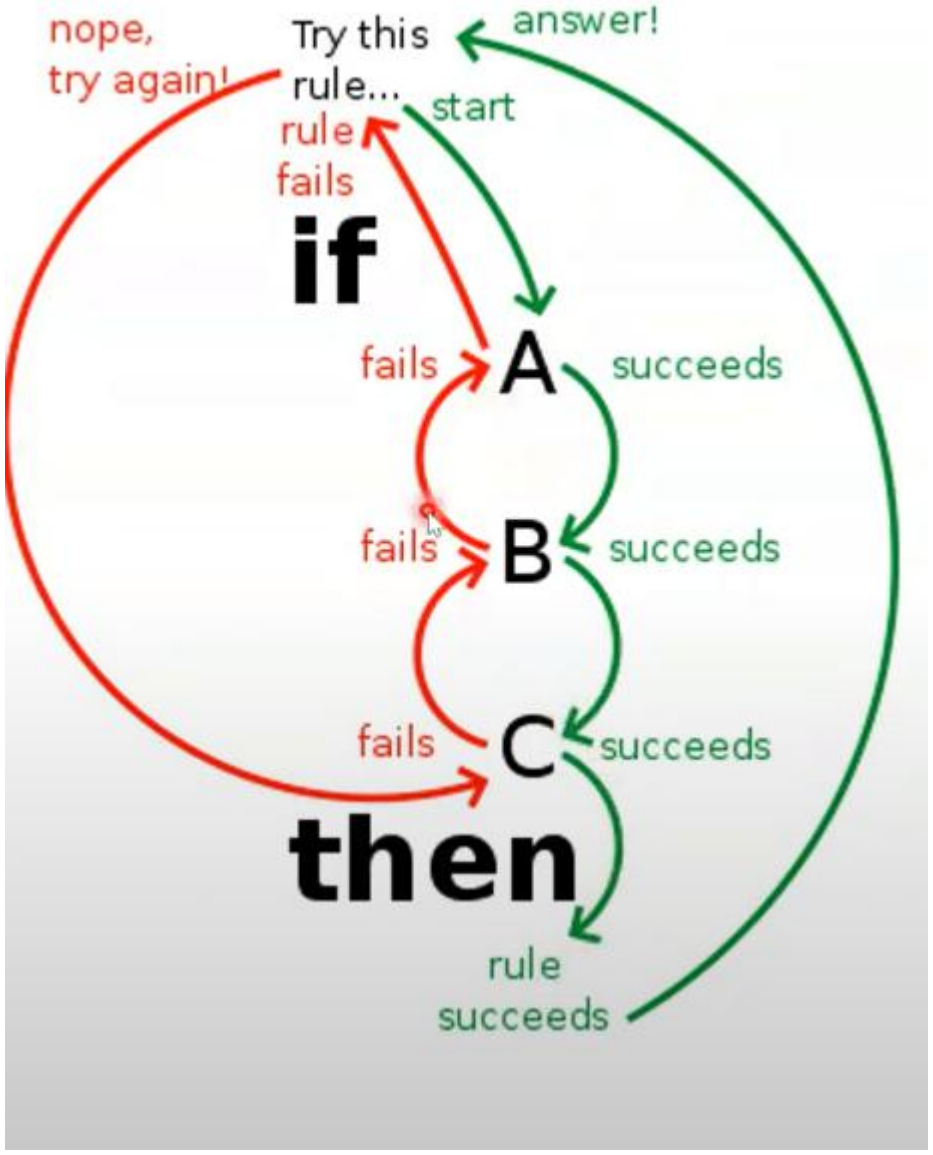
- In order to make the rule succeed, Pyke tries to match all statements with facts within the if clause through a process called **backtracking.**

# PyKE: Backtracking

While processing the list premises (A, B, C in the graph)
Within a rule's if clause:

- If succeeds at proving a premise:
  - Proceed down to the next premise in the list.
  - Trying to proceed down from the last premise in the list (when it succeeds) causes the rule to succeed.

- If fails at proving a premise:
  - Back up to the prior premise in the list and try to find another solution for it.
  - Trying to back up from the first premise in the list (when it fails) causes the rule to fail.



# PyKE: Forward Chaining Rules

- Use " Foreach", "Assert" rather than "If", "Then"

- Steps:
  1. Pyke starts with the *if* clause of the first rule and checks to see if it matches the known facts
  2. If so, it proceeds to the then clause of that rule (firing the rule) to asset a new fact.
  3. These new facts may fire other forward-chaining rules by matching their if clause.
  4. The process keeps going until no more rules could be fired.

# PyKE: Backward Chaining Rules

- Use “ Use”, “When” rather than “Then”, “If”

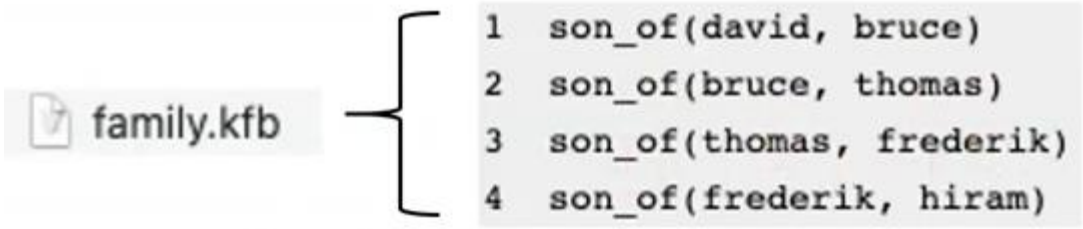
- Steps:
  1. Pyke starts by finding a rule whose then part matches the goal.
  2. Pyke then tries to prove all the subgoals in the if part of that rule
     - Some of these subgoals are matched against facts
     - Others are matched with then part of other backward-chaining rules.
  3. If all of the subgoals are proved, the rule succeeds, and the original goal is proven.

     Otherwise, the rule fails, and Pyke tries to find another rule, and so on.

# PyKE Family ES Family_relations

# Forward Chaining Rules Example-Fact Base

- **Facts** are stored in a knowledge fact base called **family.kfb** in the form:
  - son_of($son, $father)
  - Semantics: first person is the son of the second person.

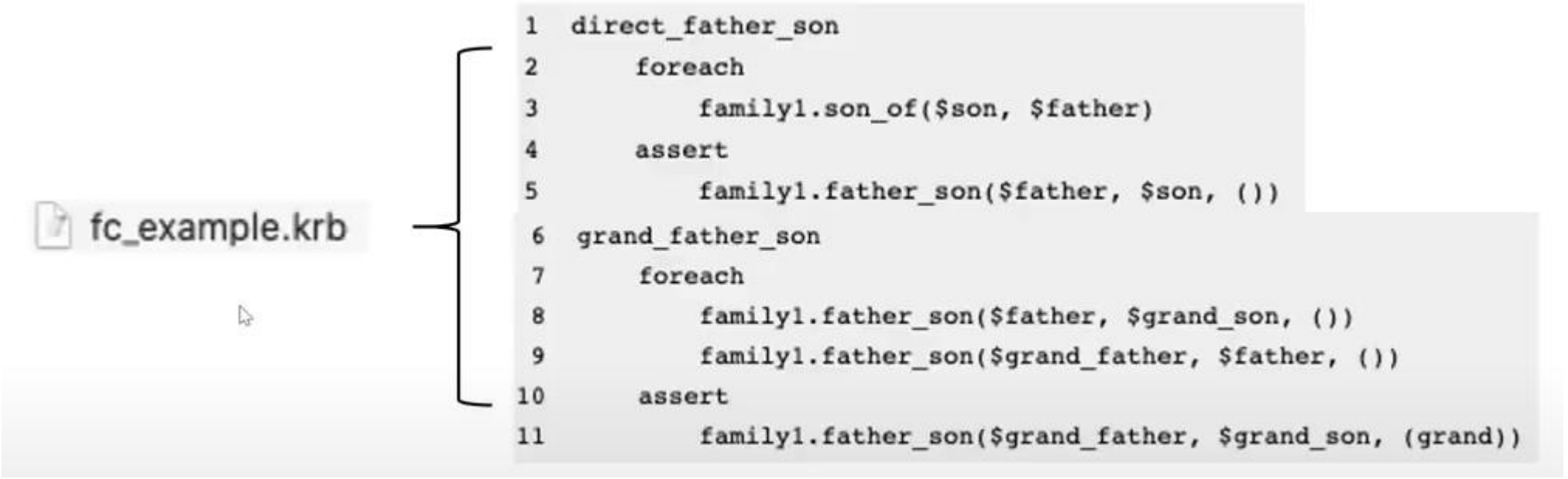



- We want to derive father-son relationships in the form :
  - father_son($father, $son, $prefix)
  - Where:

    $son: is the name of the son (e.g., david)
    $father: is the name of the father (e.g., bruce)
    $prefix: is a tuple of prefixes before the 'father' and 'son' titles to indicate the number of generations (e.g., () for direct father_son relationship, (grand), etc).

# Forward Chaining Rules Example-Face Base

- **Rules** are stored in a knowledge rule base called **fc_example.krb** in the form:



# PyKE: Forward Chaining Rules Example

- **Step 1: Direct_father_son**
- First, we establish the direct father-son relationships ($prefix is ())

```
1  son_of(david, bruce)
2  son_of(bruce, thomas)
3  son_of(thomas, frederik)
4  son_of(frederik, hiram)
```

```
1  direct_father_son
2      foreach
3          family1.son_of(david, bruce)
4      assert
5          family1.father_son($father, $son, ())
```

- When the rule fires matching line 3 to: son_of(david, bruce)
- It runs line 5 to assert: father_son(bruce, david, ())

# Backward Chaining Rules Example- Rule Base

- **Backward Chaining Rules** are stores in a knowledge rule base

```
1  direct_father_son
2      use father_son($father, $son, ())
3      when
4          family2.son_of($son, $father)

5  grand_father_son
6      use father_son($grand_father, $grand_son, (grand))
7      when
8          father_son($father, $grand_son, ())
9          father_son($grand_father, $father, ())

10 great_grand_father_son
11     use father_son($gg_father, $gg_son, (great, $prefix1, *$rest_prefixes))
12     when
13         father_son($father, $gg_son, ())
14         father_son($gg_father, $father, ($prefix1, *$rest_prefixes))
```

```
15 brothers
16     use brothers($brother1, $brother2)
17     when
18         father_son($father, $brother1, ())
19         father_son($father, $brother2, ())
20         check $brother1 != $brother2

21 uncle_nephew
22     use uncle_nephew($uncle, $nephew, $prefix)
23     when
24         brothers($uncle, $father)
25         father_son($father, $nephew, $prefix1)
26         $prefix = ('great',) * len($prefix1)

27 cousins
28     use cousins($cousin1, $cousin2, $distance, $removed)
29     when
30         uncle_nephew($uncle, $cousin1, $prefix1)
31         father_son($uncle, $cousin2, $prefix2)
32         $distance = min(len($prefixes1), len($prefixes2)) + 1
33         $removed = abs(len($prefixes1) - len($prefixes2))
```

# Backward Chaining Rules Example- Fact Base

- For this example, these are the starting set of facts stores in fact base- **family2.kfb**

```
1  son_of(tim, thomas)
2  son_of(fred, thomas)
3  son_of(bruce, thomas)
4  son_of(david, bruce)
```

- And we want to know who fred's nephew are. So we'd ask

**uncle_nephew(fred, $nepher,$prefix)**

# Step1: find a rule whose use part matches the goal

- Our goal:
  - **uncle_nephew(fred, $nephew, $prefix)**
- The first rule found by the inference engine:

```
21  uncle_nephew
22      use uncle_nephew( fred , $nephew, $prefix)
23      when
24          brothers( fred  $father)
25          father_son($father, $nephew, $prefix1)
26          $prefix = ('great',) * len($prefix1)
```

- The first subgoal to be proved:
  - **brothers(fred, $father)**

# Step2: prove the 1st subgoal in the uncle_nephew rule

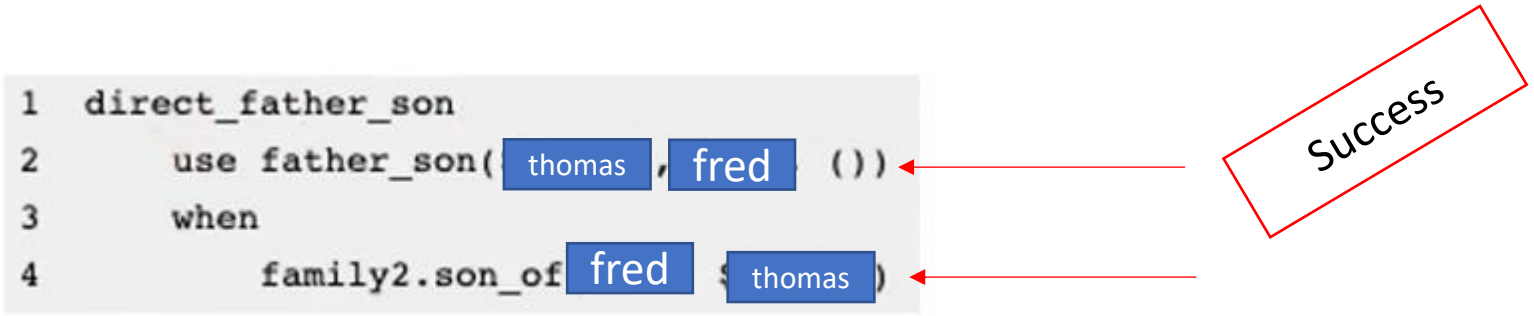
- The subgoal:
  - **brothers(fred, $father)**
- The first rule found by the inference engine:

```
15  brothers
16      use brothers( fred , $brother2)
17      when
18          father_son($father, fred , ())
19          father_son($father, $brother2, ())
20          check $brother1 != $brother2
```

- The first subgoal to be proved:
  - **father_son($father, fred,()))**

# Step3: prove the 2nd subgoal in the brother's rule

- The subgoal:
  - **father_son($father,fred, ())**
- The first rule found by the inference engine:




- The second fact in the fact base matches:
  - **Son_of(fred, thomas)**

# PyKE Weather ES Simple_bc_all

# A simple ES with ‘Questions’

- IDAT 7215 - Expert System (Weather).ipynb

- A very simple set of codes has been provided as a template for how to create your own backward chaining expert system.

- Specifically, this system decides what you should bring when walking out of the house (i.e. an umbrella, a raincoat, or nothing)

- This is in the Lab 03-Expert System folder “simple_bc_all “

# Weather Example: First Without Questions

In [1]:
```
#Run this cell
import sys
# The following command points your notebook to the location of a folder outside your working directory that you want to import.
#sys.path.append('C:/Users/Dr. Ali/Desktop/Lab 05/simple_bc_all')
sys.path.append('C:/Users/Dr. Ali//Lab 04-Expert System/simple_bc_all')

#sys.path.append('yourpath')
import driver_simple

driver_simple.bc_test()
```

```
doing proof
You should bring: raincoat

done
```

# Weather Example: First Without Questions

## Open driver_simple.py in folder simple_bc_all

```
import contextlib
import sys

from pyke import knowledge_engine
from pyke import krb_traceback

engine = knowledge_engine.engine(__file__)
```

```
def bc_test():

    engine.reset()      # Allows us to run tests multiple times.

    engine.activate('bc_simple_rules') #STUDENTS: you will need to edit the name of your rule file here

    print("doing proof")

    try:
        with engine.prove_goal('bc_simple_rules.what_to_bring($bring)') as gen: #STUDENTS: you will need to edit this line
            for vars, plan in gen:
                print("You should bring: %s" % (vars['bring'])) #STUDENTS: you will need to edit this line
    except Exception:
        # This converts stack frames of generated python functions back to the
        # .krb file.
        krb_traceback.print_exc()
        sys.exit(1)

    print()
    print("done")
    #engine.print_stats()
```

# Weather Example: Fact & Ryke KB's

bc_simple_rules.krb | facts.kfb

```
1  #Weather facts
2  raining(True)
3  windy(True)
4
5
6
7
```

data_processing_methods.py | bc_simple_rules.krb

```
1  # bc_simple_rules.krb
2
3  wear_rain_protection
4      use rain_protection(True)
5      when
6      facts.raining(True)
7
8  what_to_bring_raincoat
9      use what_to_bring(raincoat)
10     when
11     rain_protection($protection)
12     facts.windy(True)
13
14 what_to_bring_umbrella
15     use what_to_bring(umbrella)
16     when
17     rain_protection($protection)
18     facts.windy(False)
19
20 what_to_bring_nothing
21     use what_to_bring(nothing)
22     when
23     facts.raining(False)
24     facts.windy(False)
```

# Weather Example: With Questions

```
[2]: driver_simple.bc_test_questions()

doing proof
_____________________________________________________________________________
Is it raining? (y/n) y
_____________________________________________________________________________
Is it windy? (y/n) y
You should bring: umbrella
_____________________________________________________________________________
Are any of the following disasters currently occurring?
  1. Forest Fire
  2. Tornado
  3. Hurricane
  4. Pandemic
  5. None of the above
? [1-5] 1
You should bring: marshmellos

done
```

# Weather Example: With Questions

```
def bc_test_questions():

    engine.reset()      # Allows us to run tests multiple times.

    engine.activate('bc_simple_rules_questions') #STUDENTS: you will need to edit the name of your rule file here

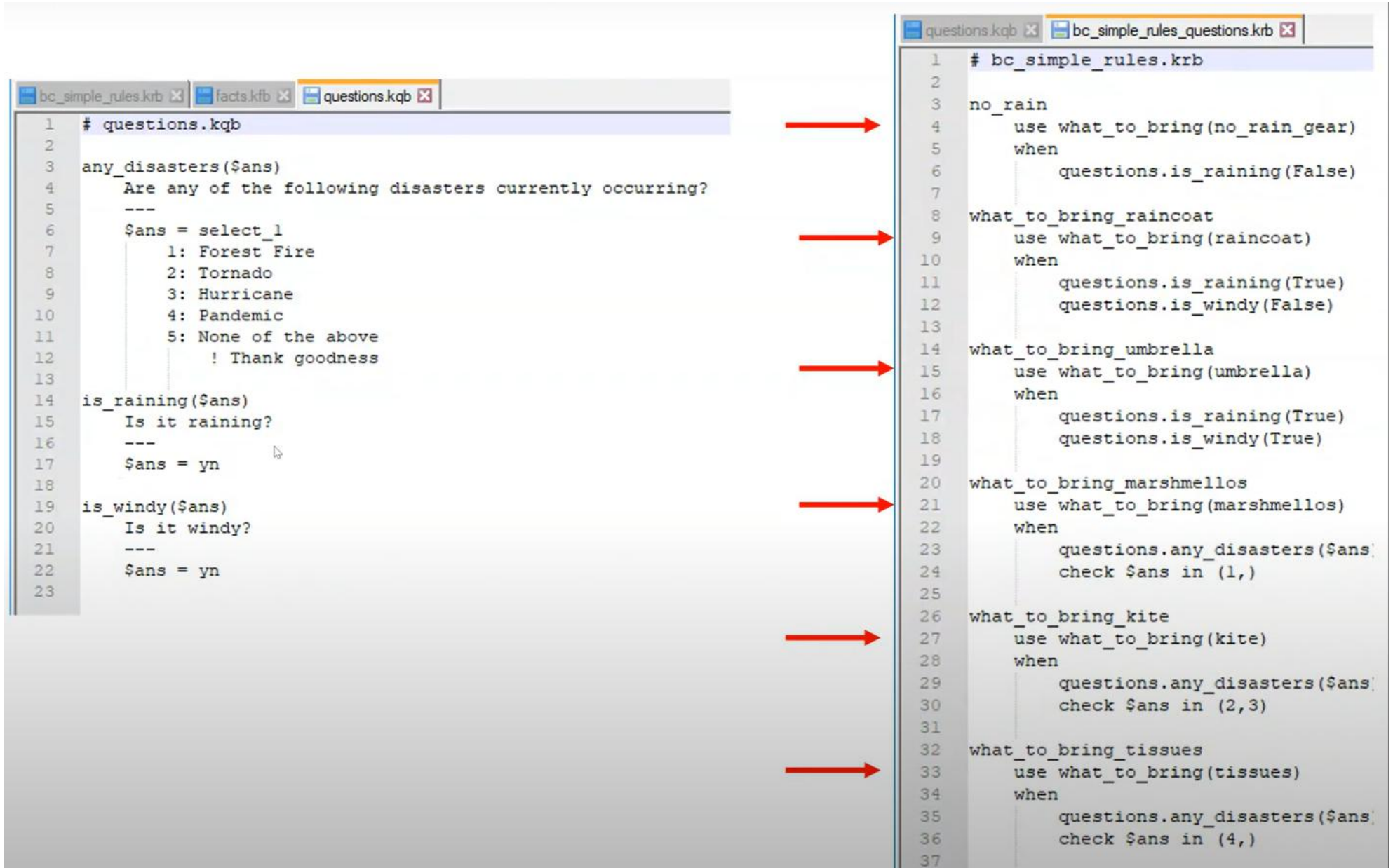
    print("doing proof")

    try:
        with engine.prove_goal('bc_simple_rules_questions.what_to_bring($bring)') as gen: #STUDENTS: you will need to edit this line
            for vars, plan in gen:
                print("You should bring: %s" % (vars['bring'])) #STUDENTS: you will need to edit this line

    except Exception:
        # This converts stack frames of generated python functions back to the
        # .krb file.
        krb_traceback.print_exc()
        sys.exit(1)

    print()
    print("done")
```

# Weather Example: Questions and Rules



bc_simple_rules.krb | facts.kfb | questions.kqb

```
# questions.kqb

any_disasters($ans)
    Are any of the following disasters currently occurring?
    ---
    $ans = select_1
        1: Forest Fire
        2: Tornado
        3: Hurricane
        4: Pandemic
        5: None of the above
            ! Thank goodness

is_raining($ans)
    Is it raining?
    ---
    $ans = yn

is_windy($ans)
    Is it windy?
    ---
    $ans = yn
```

questions.kqb | bc_simple_rules_questions.krb

```
# bc_simple_rules.krb

no_rain
    use what_to_bring(no_rain_gear)
    when
        questions.is_raining(False)

what_to_bring_raincoat
    use what_to_bring(raincoat)
    when
        questions.is_raining(True)
        questions.is_windy(False)

what_to_bring_umbrella
    use what_to_bring(umbrella)
    when
        questions.is_raining(True)
        questions.is_windy(True)

what_to_bring_marshmellos
    use what_to_bring(marshmellos)
    when
        questions.any_disasters($ans
        check $ans in (1,)

what_to_bring_kite
    use what_to_bring(kite)
    when
        questions.any_disasters($ans
        check $ans in (2,3)

what_to_bring_tissues
    use what_to_bring(tissues)
    when
        questions.any_disasters($ans
        check $ans in (4,)
```